Feature: Backups and restore

From build-plan: feature 12c

Size: heavy - two separate risks with real consequences: the live data leaving Railway every 6 hours (a way into the database, the copies locked and encrypted, the keys in GitHub), and a restore proved to bring every row back.

Branch: feature/12c-backups-and-restore

Status: verified 2026-10-23: both steps built and checked on the live database and a real copy; 12c.1 reviewed (F-437 accepted, F-438 to F-444 fixed); shared 202 tests pass.

Goal

No single mistake can lose a client's leads and bookings: a wiped Railway database, a bad migration, a bug that damages rows, or a user deleting something. Every 6 hours a GitHub job copies the live database, encrypts it and stores it in a Cloudflare R2 bucket outside Railway, where old copies thin out by themselves and none can be deleted early. A restore is proved once, end to end, and written down so it can be repeated on a bad day. Before item 13, Primo's real data.

In scope

- **The nightly job, every 6 hours**: a GitHub Actions workflow (cron, UTC) that dumps the live database through a read-only login, encrypts the dump, and uploads it to R2. A failed run sends GitHub's own failure email to Frank.
- **The schedule** (Frank, 2026-10-23), by folder in the bucket, each expiring by itself: every copy in `6h/` for 3 days; one a day also in `daily/` for 14 days; one a week also in `weekly/` for 60 days (Frank set 60 in R2, 2026-10-23); one a month also in `monthly/` for 365 days. About 40 copies at any moment.
- **Protection against one mistake**: copies encrypted with a passphrase kept in Frank's password manager, GitHub and the root `.env`; the job's database login can read but never change anything; the bucket's lock rules refuse deleting or overwriting a copy before its folder's age, even with the job's own key; the copies live outside Railway, so nothing done in Railway reaches them.
- **The restore**: one command that decrypts a copy and restores it into a local scratch `*_dev` database, then prints every table's row count; a restore of the newest real copy, compared with the live counts; a short runbook in `AGENTS.md` for the three cases (put one thing back, a damaged set of rows, the live database lost).
- **The plan split** (Frank, 2026-10-23): what happens to a business's data when it leaves, and to a customer who asks to be removed, becomes its own item, 13b.

Out of scope

- A client leaving and a customer's removal request: item 13b.
- Railway's own backups: Pro plan only; the account stays on Hobby (Frank, 2026-10-23).
- Copies more often than every 6 hours, and point-in-time recovery.
- An alert when a run never starts at all (only a failed run emails). At about 120 of 2,000 free GitHub minutes a month, running out is not a realistic cause; noted under Notes.
- Files: the app stores none yet. When it does, they go to R2 or the volume by the standards' rule, and their backup is that feature's question.
- Replacing the live database from a copy: written in the runbook, never rehearsed against live.

Build loop

Heavy, so `workflow.stepReview` is `every` for this feature: each step's plan gets Frank's yes just before it is built. **After the green light, nothing stops until the review**: build, checks, tick the box, write the build log, commit and push to `feature/12c-backups-and-restore`, `/audit` scoped to the step, then the independent review. Frank's outside trip (Railway, Cloudflare, GitHub) is one planned trip inside 12c.1, listed in its plan, so there is no surprise trip. The only early stops are the three in `AGENTS.md`. Blocking findings (P0/P1) are fixed by default; P2/P3 are recorded and carried. `/complete` makes the final pass and merges on Frank's yes.

Build steps

- ☐ **12c.1 A copy lands in R2 every 6 hours.** "Every 6 hours a locked, encrypted copy of the live database appears in the R2 bucket, in the right folders, and nobody can delete it early."
  - Frank's one trip, in this order: 1. Railway: turn on the Postgres service's public TCP proxy (Settings, Networking). 2. Cloudflare: create the bucket `scheduleads-backups`; its lifecycle rules (`6h/` 3 days, `daily/` 14, `weekly/` 60, `monthly/` 365) and bucket lock rules with the same prefixes and ages; an R2 API token with Object Read & Write on that bucket only. 3. His password manager: a new long passphrase, "Scheduleads backup key". 4. GitHub, the `scheduleads` repo's Actions secrets: `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `BACKUP_PASSPHRASE` (typed by him; they never pass through chat).
  - On his yes, through the tunnel: the read-only login `backup_reader` (a generated password, `GRANT pg_read_all_data`), and its connection string through the public address saved as the secret `BACKUP_DATABASE_URL` with `gh secret set`, piped, never printed.
  - Code: `.github/workflows/database-backup.yml`: `schedule: cron "0 /6 "` and `workflow_dispatch`; installs `postgresql-client-18` from the PostgreSQL apt repository; `pg_dump --format=custom`; refuses a dump under 10 KB or with no tables; `gpg --symmetric --cipher-algo AES256` with the passphrase from the secret; uploads with the runner's `aws` CLI to

R2's S3 endpoint as `6h/<UTC yyyy-mm-ddTHHMM>Z.dump.gpg`, plus `daily/`, `weekly/`, `monthly/` on the matching runs; prints the copy's size and table count, never data.
    - `AGENTS.md`: a Backups paragraph (where, the schedule, the secrets' names, the login, how to run a copy now). **Done when:** a run by hand and the next scheduled run each put a copy in R2 under the right folders; the downloaded copy is unreadable without the passphrase; an `INSERT` through `backup_reader` is refused; deleting a fresh copy with the job's own key is refused; the copy's size is written in the log.
- [x] **12c.2 A copy comes back, every row.** "Frank downloads the newest copy, one command restores it into a scratch database on the laptop, and every table holds what the live database held."
    - Code: `packages/shared/scripts/restore-backup.ts` (`npm run backup:restore -- workspace=@scheduleads-app/shared -- <copy.dump.gpg> --into <name>`): the target must be a local database ending in `_dev` and must not be the one `DATABASE_URL` names, so the working dev database is never replaced; it is dropped and made fresh, the copy decrypted with `gpg` (which asks for the passphrase), restored with `pg_restore --no-owner --no-privileges`, and every table's row count printed. Finds `pg_restore` on `PATH` or in `PG_BIN` (on this laptop `C:\Program Files\PostgreSQL\18\bin`).
    - The drill: the newest copy restored into `scheduleads_restore_dev`, its counts compared with the live counts read through `backup_reader`.
    - `AGENTS.md`: the restore runbook, the three cases. **Done when:** the target guard's tests pass; the newest real copy restores with no error and every table's count matches the live database, or differs only by rows added after the copy's time, each written in the log; IA a la Orden's business, people and bookings are there.

Files / areas

- `.github/workflows/database-backup.yml` (new; the repo has no workflows yet)
- `packages/shared/scripts/restore-backup.ts` (new), its target guard beside it with its test, `packages/shared/package.json` (`backup:restore`)
- `packages/shared/helpers/assert-local-dev-database.ts` (reused, unchanged)
- `AGENTS.md` (Backups, the restore runbook)
- `blueprint/build-plan.md`, `blueprint/project-plan.md`, `blueprint/context/project-overview.md` (the split: 12c is backups and restore, 13b is leaving and removal)
- Outside the repo, Frank's: Railway (the TCP proxy), Cloudflare (bucket, rules, token), GitHub (secrets), his password manager (the passphrase)

Data / contracts

- **A copy**: `pg_dump --format=custom` of the whole live database, encrypted with GnuPG symmetric AES-256; object key `<folder>/<UTC yyyy-mm-ddTHHMM>Z.dump.gpg` (minutes, so two runs never

share a name the lock would refuse to overwrite); folders `6h`, `daily`, `weekly`, `monthly`. The same copy may sit in up to four folders.

- **Which folders**: every run `6h/`; also `daily/` when the newest there is not from today (UTC), `monthly/` when the newest there is not from this month, and `weekly/` when the newest there is 7 days old or more. What the bucket holds decides, not the clock (F-439).
- **Expiry and lock**: per folder, 3, 14, 60 and 365 days; R2 lifecycle deletes, the lock refuses any earlier delete or overwrite.
- **`backup_reader`**: a login role with `pg_read_all_data` and `BYPASSRLS` (graphile-worker's four `_private_*` tables use row-level security, which `pg_dump` will not read around otherwise); no write, no DDL. Its password exists only in the `BACKUP_DATABASE_URL` secret; lost means rotated.
- **`backup:restore <copy> --into <name>`**: refuses (message, exit 1) a target that is not local, does not end in `_restore_dev`, or comes from a `DATABASE_URL` with options after its name (F-445, F-446); prints the target first; prints `table: rows` for every table after.
- Secrets in GitHub: `BACKUP_DATABASE_URL`, `BACKUP_PASSPHRASE`, `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`. The passphrase also in Frank's Google Password Manager and the root `.env` (`BACKUP_PASSPHRASE`, for the restore; Frank, 2026-10-23): without it no copy can ever be opened.

Testing

`npm run test --workspace=@scheduleads-app/shared`. One test per rule that would hurt if broken:

- 12c.2: the restore target guard refuses a non-local database, a name not ending in `_restore_dev` (the working database and another project's `_dev` one included), and a `DATABASE_URL` with options after its name, and accepts `scheduleads_restore_dev`.
- 12c.1 has no app logic; its checks are observed (the runs, the refused insert, the refused delete, the unreadable download) and written in the log.

Decisions (made while planning, Frank's to overrule)

1. **Copies outside Railway, in R2** (Frank, 2026-10-23): Railway's backups need Pro and die with the disk; R2's free 10 GB holds about 40 copies of about 1 MB today with room for years. 2. **Stay on Hobby** (Frank, 2026-10-23): Pro's backups add nothing the outside copy lacks; upgrade when the WhatsApp app goes on Railway and the $20 credit covers both. 3. **A GitHub job runs it** (Frank, 2026-10-23), not the API's job runner (inside Railway, would need an S3 package and dies with the API) nor a Railway cron service (inside Railway). 4. **Every 6 hours, then daily, weekly, monthly** (Frank, 2026-10-23): hourly lost to GitHub's free minutes for little gain; at most 6 hours lost in a disaster. 5. **A public address with a read-only login** (Frank, 2026-10-23), not a Railway token through the tunnel: a leaked read-only login can read but never delete; a leaked Railway token could wipe the project. 6. **Encrypted with GnuPG**, already on GitHub's runners and in Git for Windows on the laptop, so nothing is installed; rejected: age (a new tool on

both ends), no encryption (every client's customers in a file). 7. **Bucket lock rules** so a copy outlives a leaked key or a wrong click: the job's key can add copies but no key can remove one early. 8. **Expiry by folder in R2**, not code that deletes: nothing to break, nothing to run. 9. **The restore goes into a local scratch database**, never the working dev one and never live; replacing live is a runbook entry, done only on Frank's yes on the day.

Amendments

- 2026-10-23, 12c.1: weekly copies kept 60 days, not 90 (Frank set 60 in R2 and kept it).
- 2026-10-23, 12c.1: `backup_reader` also has `BYPASSRLS`. The first run failed: graphile-worker's `_private_job_queues`, `_private_jobs`, `_private_known_crontabs` and `_private_tasks` have row-level security on, and `pg_dump` refuses to copy them past it. Reading only; an insert is still refused (`42501`).
- 2026-10-23, 12c.1: object names carry minutes and `Z` (`2026-10-23T1556Z.dump.gpg`): the lock refuses an overwrite, so two runs in one hour must never share a name.
- 2026-10-23, 12c.1: no temporary `push` trigger: `gh workflow run database-backup.yml --ref <branch>` runs the job from the feature branch, so it was removed before the step closed.
- 2026-10-23, 12c.1: one copy is 105 KB before locking and 24 KB locked (GnuPG compresses), not about 1 MB: about 40 copies hold about 1 MB of R2.
- 2026-10-23, 12c.2: the restore reads the key from `BACKUP_PASSPHRASE` in the root `.env` (Frank keeps it there) and asks only when it is missing; it finds `pg_restore` and `gpg` in their Windows installs when they are not on PATH (`PG_BIN` and `GPG` override).
- 2026-10-23, 12c.2: the drill's live counts were read with read-only queries through the superuser, not `backup_reader`, whose address lives only in the GitHub secret. All 33 tables matched.
- 2026-10-23, 12c.2 review: the target must end in `_restore_dev`, not just `_dev`, so another project's database on this laptop is never dropped (F-445); a `DATABASE_URL` with a query string is refused and `pg_restore` gets the address as separate flags with no other `PG*` setting (F-446); both tools are checked before anything is dropped (F-447); leftover opened copies are removed at the start and the runbook says to drop the scratch database (F-448).

Notes for the AI

- Scheduled workflows run only from the default branch, so the schedule starts at the merge. 12c.1 proves the job by hand: check whether `gh workflow run database-backup.yml --ref feature/12c-backups-and-restore` works before the file is on `main`; if not, a temporary `push` trigger on this branch and this file only, removed before `/complete`. The "next scheduled run" in 12c.1's Done when is then seen after the merge and written in the log.
- GitHub sends the failure email to whoever last changed the cron line; check Frank's Actions notification setting is on during his trip.

- The R2 token is Object Read & Write; the lock rules, not the token, are what refuse deletes.
- `pg_dump` must be version 18 or newer against Postgres 18; Ubuntu's own client is older.
- No secret value is ever printed: not in the workflow log, not in chat, not in the build log.
- Without a payment method GitHub stops workflows when the free minutes run out; at about 120 a month that is far off, but a run that never starts sends no email.
- Never stage `blueprint/ai-voice-proposal.md`.

Implementation walkthrough

Feature 12c was specced heavy, two steps, and kept them; each step got its own independent review, then one final review over the whole feature passed. 12c.1 is `a9e3d66` (the job) and `0dd870b` (verified, the temporary push trigger dropped), its review fixes `c8ea613`, F-437 accepted in `e445997`. 12c.2 is `9cfdd92` and `d1f883d` (formatting), its review fixes `f1eda6f`, the config formatted in `7208529`. The branch was cut from `chore/plan-default-sender`, so its one plan commit (`8640e41`, item 25 sends booking emails from `booking-crm.com` by default) merges with it.

The plan split

12c became backups and restore only; what happens when a client leaves, or a customer asks to be removed, became item 13b (Frank, 2026-10-23). The account stays on Railway's Hobby plan, whose own backups need Pro and die with the disk anyway.

Frank's trip (12c.1.1)

Railway: the Postgres service's public TCP proxy, applied together with his earlier, never-applied switch of the API to `main` during Railway's platform outage (the API rebuilt from `24ba21a`, healthy). Cloudflare: the bucket `scheduleads-backups`, lifecycle and bucket lock rules per folder (`6h/` 3 days, `daily/` 14, `weekly/` 60 by Frank's choice, `monthly/` 365), and an R2 token with Object Read & Write on that bucket only, rolled once because Cloudflare shows the secret only at creation. The passphrase was generated straight onto the clipboard and lives in Google Password Manager, the GitHub secret and the root `.env`. GitHub: five secrets.

The read-only login (12c.1.2)

`backup_reader`, made on the live database over the proxy with SSL: `pg_read_all_data` and, after the first run failed, `BYPASSRLS`, because graphile-worker's four `_private_*` tables use row-level security and `pg_dump` refuses to copy past it. It reads, and an insert is refused with `42501`. Its address went into `BACKUP_DATABASE_URL` with `gh secret set`, never printed.

The job (`.github/workflows/database-backup.yml`)

Every 6 hours (`0 /6`) and on demand: it checks the PostgreSQL apt key's fingerprint, installs the PostgreSQL 18 client, runs `pg_dump --format=custom` with `PGSSLMODE=require`, refuses a copy under 10 KB or with no tables (counting `TABLE` lines, not `TABLE DATA`), locks it with `gpg --symmetric --cipher-algo AES256` (the passphrase on a file descriptor), and uploads it with the runner's `aws` CLI. The folders come from what the bucket already holds, not the clock: `daily/` when the newest there is not from today (UTC), `monthly/` when not from this month, `weekly/` when the newest is 7 days old, so a dropped or late run never skips one nor doubles one. Names carry minutes (`2026-10-23T1628Z.dump.gpg`) because the lock refuses an overwrite. Runs by hand from the feature branch work (`gh workflow run --ref`), so no temporary trigger was needed. One copy is 105 KB, 24 KB locked; 33 tables.

The restore (`packages/shared/backups/` , `scripts/restore-backup.ts`)

`restoreTargetUrl` accepts only a database on this laptop whose name ends in `_restore_dev`, from a `DATABASE_URL` with nothing after its name, so neither the working database nor another project's is ever dropped. `backup:restore` checks `gpg` and `pg_restore` exist, removes leftovers of a stopped run, opens the copy with `BACKUP_PASSPHRASE` from `.env` (on stdin), drops and makes the target fresh, runs `pg_restore --no-owner --no-privileges` with the address as separate flags and no other `PG*` variable, deletes the opened copy, and prints every table's rows. The drill: the 15:56 copy restored into `scheduleads_restore_dev`, all 33 tables matched live row for row, IA a la Orden with its person, booking and both logins; the scratch database was dropped after.

Written down

`AGENTS.md` : the Backups paragraph and the restore runbook (one row back, damaged rows, the live database lost). The build log gained a fourth Project tab, Selling points, holding what 12c gives a client (copies every 6 hours, kept a year, outside the host and encrypted, not deletable early).

Carried

F-436 (the `.env.example` live list, from 10b), F-444 and F-449 (fixed, waiting on a later review pass), F-450 (feature and finding tags in new comments), F-451 (the overview fingerprint, recomputed by this `/complete`), F-452 (`backups/` missing from the package's editor tsconfig), F-453 (the runbook's `gpg ... > copy.dump` breaks in PowerShell 5.1), F-454 (`.env.example` lacks `BACKUP_PASSPHRASE` , `PG_BIN` , `GPG`). The first scheduled run, and the first expiry under a lock of the same age, happen after the merge.

Findings

12c/F-437 [P2] accepted - The public TCP proxy opened for the backup lets every live login in from the internet, not only backup_reader

File: blueprint/context/current-feature.md:156-158 (Decision 5); AGENTS.md:556-559 **Found:** 2026-10-23 by independent review of step 12c.1 (scope: 8640e41..0dd870b; lenses: quality, security, performance, tests) **Why it matters:** Decision 5 weighs only the job's own credential ("a leaked read-only login can read but never delete"). Railway's TCP proxy forwards raw Postgres traffic and authenticates by password, so the address it opened also accepts the `postgres` superuser that the API's `DATABASE_URL` uses (Railway publishes exactly that pair as `DATABASE_PUBLIC_URL`). Before 12c.1 that password alone was useless outside Railway's private network or a Railway-token tunnel; now anyone holding it (the laptop's root `.env` carries the Railway line for the tunnel) can write to and drop the live database from anywhere. The password is long and random, so this is a leak risk, not a brute-force one. Missing validation: whether the superuser connects through the proxy address (a `psql` through it with the Railway credentials, by Frank). **Suggested fix:** Record the trade in Decision 5 and the AGENTS.md Backups paragraph so it is Frank's call with the full picture. If he wants it narrowed: give the API its own non-superuser login so the superuser password lives nowhere in daily use, or rotate the superuser password and keep it only in the password manager. **Resolution:** Confirmed 2026-10-23: the one-off that made backup_reader connected as the live superuser (`postgres`, from Railway's variables) through `shortline.proxy.rlwy.net` over SSL, so every live login is now reachable from the internet with its password. Open: Frank's decision (keep the proxy and rotate or lock down the superuser, or another way in). Accepted by Frank, 2026-10-23: a 32-character random password plus an encrypted connection; rotate it if the `.env` ever leaks.

12c/F-438 [P3] closed - The job's "tables" count counts every table twice, so "66 tables" in the log and build log is about 33

File: .github/workflows/database-backup.yml:57-58 **Found:** 2026-10-23 by independent review of step 12c.1 (scope: 8640e41..0dd870b; lenses: quality, security, performance, tests) **Why it matters:** `pg_restore --list` writes one `TABLE` entry and one `TABLE DATA` entry per table, and `grep -c " TABLE "` matches both. The migrations create 28 tables, plus drizzle's ledger and graphile-worker's five, about 33 to 34, against the 66 the run printed and the build log repeats ("opens all 66 tables"). The refusal still works (zero is zero), but the spec's contract "prints the copy's size and table count" prints the wrong number, and 12c.2 compares per-table counts, where 66 against roughly 33 restored tables reads as missing tables. **Suggested fix:** Count only the definition entries, e.g. `grep " TABLE " | grep -vc " TABLE DATA "` (keeping `|| true`), and correct the count in the 12c.1 log entry. **Resolution:** Fixed 2026-10-23: the count drops `TABLE DATA` lines. Run 39987049465 printed `33 tables` for the same copy that read 66 before. Closed 2026-10-23 by the independent review of 24ba21a..7208529: line 66 counts `TABLE` entries without `TABLE DATA`; `grep -vc` with `|| true` still yields 0 for an empty list, so the refusal holds.

12c/F-439 [P3] closed - A dropped or very late 00 UTC run leaves that day with no daily copy, and a Sunday or the 1st with no weekly or monthly one, without any email

File: .github/workflows/database-backup.yml:69-77 **Found:** 2026-10-23 by independent review of step 12c.1 (scope: 8640e41..0dd870b; lenses: quality, security, performance, tests) **Why it matters:** Only the

run whose start hour is 00-05 UTC adds `daily/`, `weekly/` or `monthly/`. GitHub documents that scheduled runs can be delayed under load and that queued scheduled runs can be dropped; a dropped 00 run is a run that never starts, so no failure email. That day's only copies then sit in `6h/` and expire after 3 days; a dropped first-of-month run means no copy of that month survives past 14 days. The opposite edge also exists: an 18 UTC run delayed past midnight lands in slot 0 and adds a second `daily/` (and on a Sunday or the 1st, a second weekly or monthly) copy, which the lock then keeps. The slot comment ("a late start still counts") covers delays under 6 hours only. **Suggested fix:** Decide by what is already stored rather than by the hour: before uploading, list `daily/<today's UTC date>` and add `daily/` when nothing for today is there yet (and the same for the ISO week in `weekly/` and the month in `monthly/`). The first successful run of a day, week or month is then the kept one, and the double copy goes. Or accept it and say so in the spec's Out of scope. **Resolution:** Fixed 2026-10-23: the folders come from what the bucket holds (newest `daily/` not today, newest `monthly/` not this month, newest `weekly/` 7 days old or more), not from the start hour. Run 39987049465 (16:28 UTC) stored 6h, daily, weekly and monthly; run 39987173568 a minute later stored 6h only. Spec Data / contracts and AGENTS.md updated. Closed 2026-10-23 by the independent review of 24ba21a..7208529: lines 80-93 choose `daily/`, `weekly/` and `monthly/` from the newest name already in each folder; an empty or unreadable folder adds the copy (more copies, never fewer).

12c/F-440 [P3] closed - The workflow does not require an encrypted connection; the live data travels however the secret URL says

File: .github/workflows/database-backup.yml:37,53 **Found:** 2026-10-23 by independent review of step 12c.1 (scope: 8640e41..0dd870b; lenses: quality, security, performance, tests) **Why it matters:** The dump crosses the public internet through the TCP proxy. The workflow sets no `PGSSLMODE`, so `pg_dump` uses whatever `sslmode` the `BACKUP_DATABASE_URL` secret carries, and libpq's default `prefer` falls back to plaintext if the server's TLS offer is stripped on the way, sending the backup_reader password and every client's rows in the clear. The build log says the login was made "over the new public address (SSL)", which shows the server offers TLS, not that the secret insists on it. Missing validation: whether the secret's URL carries `sslmode=require` (or stronger); it cannot be read from the repo. **Suggested fix:** Add `PGSSLMODE: require` to the step's `env` (libpq uses it whenever the URL names no `sslmode`), so the rule lives in the reviewed file rather than only in a secret. `require` stops passive reading; Railway's self-signed certificate rules out `verify-full` without its CA. **Resolution:** Fixed 2026-10-23: `PGSSLMODE: require` in the step env (the secret URL also carries `sslmode=require`). Closed 2026-10-23 by the independent review of 24ba21a..7208529: `PGSSLMODE: require` at line 49 applies whenever the secret URL names no sslmode.

12c/F-441 [P3] closed - The PostgreSQL apt signing key is downloaded on every run and trusted without checking its fingerprint

File: .github/workflows/database-backup.yml:27-31 **Found:** 2026-10-23 by independent review of step 12c.1 (scope: 8640e41..0dd870b; lenses: quality, security, performance, tests) **Why it matters:** The job that holds the database login, the passphrase and the R2 key installs and runs `pg_dump` from a repository

whose trust root is whatever `www.postgresql.org` serves at that moment. This is PGDG's own published recipe and HTTPS protects it, so the risk is small, but a pinned fingerprint costs one line and turns a swapped key into a failed run instead of trusted packages. **Suggested fix:** After the download, check `gpg --show-keys --with-colons` on the file for PGDG's documented fingerprint for ACCC4CF8 (`B97B0AFCAA1A47F044F244A07FCC7D46ACCC4CF8`) and exit 1 if it is not there. **Resolution:** Fixed 2026-10-23: the downloaded key is trusted only if its fingerprint is `B97B0AFCAA1A47F044F244A07FCC7D46ACCC4CF8` ; runs 39987049465 and 39987173568 installed the client past the check. Closed 2026-10-23 by the independent review of 24ba21a..7208529: lines 30-36 compare the downloaded key's primary fingerprint with PGDG's published one and exit 1 otherwise.

12c/F-442 [P3] closed - AGENTS.md says backup_reader has pg_read_all_data "nothing else", but it also bypasses row-level security

File: AGENTS.md:558 **Found:** 2026-10-23 by independent review of step 12c.1 (scope: 8640e41..0dd870b; lenses: quality, security, performance, tests) **Why it matters:** The spec's Data / contracts and the 12c.1 amendment record `BYPASSRLS` as part of the login (graphile-worker's `_private_*` tables need it). AGENTS.md, the file every session loads and the one a rebuild of the login would follow, still says "nothing else", so a login recreated from it would make the next run fail on those four tables. **Suggested fix:** Say `pg_read_all_data` and `BYPASSRLS` (reading only), with the reason in a few words. **Resolution:** Fixed 2026-10-23: AGENTS.md names `pg_read_all_data` and `BYPASSRLS` and why. Closed 2026-10-23 by the independent review of 24ba21a..7208529: AGENTS.md:558-560 names `pg_read_all_data` and `BYPASSRLS` and why.

12c/F-443 [P3] closed - The backup passphrase is also in the laptop's root .env, where the spec and AGENTS.md say it lives only in GitHub and the password manager

File: AGENTS.md:564-567; blueprint/context/current-feature.md:30-31,133-135 **Found:** 2026-10-23 by independent review of step 12c.1 (scope: 8640e41..0dd870b; lenses: quality, security, performance, tests) **Why it matters:** The 12c.1 build log says the backup key went into "Google Password Manager, GitHub and .env", and the download check opened the copy with "your key from .env". The root `.env` is loaded into every local API, test and script process and is copied to each machine, and it already holds the live Railway login; one leak of that file now also opens every backup copy. The docs promise the opposite, and `.env.example` has no line for it. 12c.2's restore plan has `gpg` ask for the passphrase, so nothing needs it in `.env` . **Suggested fix:** Remove it from `.env` (the password manager is the second copy), or, if Frank wants it there, say so in the spec's protection line and AGENTS.md and add the name to `.env.example` . **Resolution:** Fixed 2026-10-23 in the docs: the spec and AGENTS.md now say the passphrase is also in the root `.env` (Frank's choice that day, for the restore) and in Google Password Manager. The risk that one leaked `.env` opens the copies is left to Frank (that file already holds the live database tunnel line). Note from the independent review of 12c.2 (e445997..d1f883d): the restore now reads `BACKUP_PASSPHRASE` from the root `.env` and passes it to gpg on stdin, never in arguments or logs; the runbook and spec amendment say so, consistent with this resolution. Status left for the reviewer of a pass that re-reviews 12c.1's files. Closed 2026-10-23 by the independent review of 24ba21a..7208529:

AGENTS.md:569-571 and the spec (lines 30-31, 134-137) say the passphrase is also in the root `.env`, and the restore hands it to gpg on stdin only. The missing `.env.example` line is recorded separately as F-454.

12c/F-445 [P3] closed - The restore drops any local _dev database it is named, another project's working one included, without asking

File: packages/shared/backups/restore-target.ts:13-23; packages/shared/scripts/restore-backup.ts:83 **Found:** 2026-10-23 by independent review of step 12c.2 (scope: e445997..d1f883d; lenses: quality, security, performance, tests) **Why it matters:** The guard protects only the one database `DATABASE_URL` names. The same local server holds other projects' working databases: `list-board`'s `.env.example` points at `localhost:5432/list_board_dev`, and `restoreTargetUrl(<local scheduleads_dev URL>, "list_board_dev")` returns a target (checked against the built guard). The script then runs `drop database ... with (force)`, cutting off that project's running server, with no prompt. A wrong name from shell history or a copied command is enough. The spec's aim is "a restore never replaces real work". **Suggested fix:** Accept only names ending in `_restore_dev` (the runbook's `scheduleads_restore_dev` already does) and add that rule to the guard's tests; the error names the expected shape. **Resolution:** Fixed 2026-10-23: the target must end in `_restore_dev`; `list_board_dev` is refused (test, and by hand: one-line refusal, exit 1). Closed 2026-10-23 by the independent review of 24ba21a..7208529: restore-target.ts:11 accepts only `^[a-z0-9_]+_restore_dev$`, tested at restore-target.test.ts:23 (another project's `_dev` database refused).

12c/F-446 [P3] closed - The restore target keeps the working URL's query string and goes to pg_restore as a URL, password included, on its command line

File: packages/shared/backups/restore-target.ts:21-24; packages/shared/scripts/restore-backup.ts:90-92 **Found:** 2026-10-23 by independent review of step 12c.2 (scope: e445997..d1f883d; lenses: quality, security, performance, tests) **Why it matters:** The guard checks only the URL's host and path, but `target` is the whole working URL with its path swapped: `?port=5433&dbname=railway` survives (checked against the built guard). libpq, which pg_restore uses, reads `host`, `hostaddr`, `port` and `dbname` from the query and they win over the host and path; it also takes `PGHOSTADDR` from the environment, which postgres.js ignores and which the root `.env` would put there. Today such a URL fails earlier only by accident (postgres.js sends unknown query keys as startup parameters and the server refuses them), so three parsers read one URL three ways and only one is guarded. Separately, `--dbname <url>` puts the local Postgres password in pg_restore's process arguments, visible to process listings for as long as the restore runs. **Suggested fix:** In `restoreTargetUrl`, refuse a `DATABASE_URL` with a query or fragment (or clear both) and test it; call pg_restore with `--host`, `--port`, `--username` and `--dbname <name>`, the password in `PGPASSWORD` in its env, and `PGHOSTADDR` / `PGSERVICE` removed from that env. **Resolution:** Fixed 2026-10-23: a `DATABASE_URL` with a query string or hash is refused (test); `pg_restore` gets `--host`, `--port`, `--username`, `--dbname` as separate flags with the password in `PGPASSWORD` and every other `PG` *variable removed from its environment. The re-run drill restored the same 33 tables. Closed 2026-10-23 by the independent review of 24ba21a..7208529: restore-target.ts:21-25 refuses a query or*

hash (tested at restore-target.test.ts:38); restore-backup.ts:116-136 passes separate flags, the password in `PGPASSWORD`, and strips every other `PG` variable from pg_restore's environment.

12c/F-447 [P3] closed - A gpg or pg_restore that cannot be found is reported as "wrong key" or as restore errors

File: packages/shared/scripts/restore-backup.ts:74-75,97-99 **Found:** 2026-10-23 by independent review of step 12c.2 (scope: e445997..d1f883d; lenses: quality, security, performance, tests) **Why it matters:** When the program is missing, `spawnSync` returns `status: null` with `error.code = "ENOENT"` and prints nothing. The script then says "The copy did not open: wrong key, or not a backup copy." for gpg, and for pg_restore "ended with errors (exit none)" followed by "0 tables.", after the target was already dropped. On a second machine (Git or PostgreSQL 18 not at the default path, which the workspace expects) the first message sends Frank hunting for a passphrase that is fine, during a restore he may need in a hurry. **Suggested fix:** Check `opened.error` / `restored.error` first and print "gpg was not found: put it on PATH or set GPG" (and the same for pg_restore with `PG_BIN`); better, check both programs exist before anything is decrypted or dropped. **Resolution:** Fixed 2026-10-23: `gpg` and `pg_restore` are each run with `--version` before anything is opened or dropped; a missing one is named with the setting that points at it. Closed 2026-10-23 by the independent review of 24ba21a..7208529: restore-backup.ts:55-63 runs both tools with `--version` before anything is opened or dropped and names the setting that points at a missing one.

12c/F-448 [P3] closed - The opened copy is left in the temp folder on Ctrl+C, and the scratch database keeps every client's customers with no step to remove it

File: packages/shared/scripts/restore-backup.ts:48,124; AGENTS.md:573-580 **Found:** 2026-10-23 by independent review of step 12c.2 (scope: e445997..d1f883d; lenses: quality, security, performance, tests) **Why it matters:** `finally` removes the decrypted dump on every thrown error, but Node's default Ctrl+C exits without running it, and `spawnSync` blocks while gpg or pg_restore runs, so an interrupted restore leaves the whole unencrypted database in `%TEMP%\scheduleads-restore-`, *against the comment "the opened copy never stays on disk". More lasting: `scheduleads_restore_dev` holds the full live data after every restore and the runbook never says to drop it when done, while the copies themselves are encrypted precisely because they hold every client's customers (Decision 6).* ***Suggested fix:*** *Add one runbook line: when the rows are found, drop `scheduleads_restore_dev`. At the start of a run, remove leftover `scheduleads-restore-`* folders in the temp folder (or soften the comment). **Resolution:** Fixed 2026-10-23: leftover `scheduleads-restore-` *folders in the temp folder are removed at the start of each run; the runbook says to drop `scheduleads_restore_dev` when done (it was dropped after the drill). Closed 2026-10-23 by the independent review of 24ba21a..7208529: restore-backup.ts:66-70 removes leftover `scheduleads-restore-`* folders at the start of each run, and AGENTS.md:584-585 says to drop the scratch database when done. A Ctrl+C still leaves the opened copy until the next restore, which is the option the suggested fix allowed.

Independent review

The final receipt, over the whole feature. Each step also had its own independent review (12c.1: F-437 to F-444; 12c.2: F-445 to F-449), recorded in the findings above and in the build log.

Status: passed **Target commit:** 7208529fb5e5112bba3370efb3e1506d6e064ec1 **Base commit:** 24ba21a4d599efcdca0c3a918706e65221b19bf8 **Base ref:** main **Spec hash:** 704d4f66243a1278149916e218b34127f27acc6e8dd9aedb62c7a42ce052ad76 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-23T17:03:50Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-23T17:10:52Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete `24ba21a4d599efcdca0c3a918706e65221b19bf8..7208529fb5e5112bba3370efb3e1506d6e064ec1` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `npm run test --workspace=@scheduleads-app/shared` : pass (30 files, 203 tests)
- `npm run test --workspace=backend` : pass (91 files, 1052 tests, local Postgres reachable)
- `npm run test --workspace=@frankdmosquera/booking-component` : pass (16 files, 83 tests)
- `npm run build` : pass (backend tsc)
- `npm run build --workspace=frontend` : pass
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass
- `tsc --ignoreConfig --noEmit` (package options) on `backups/restore-target.test.ts` and `scripts/restore-backup.ts` : pass
- `/check` : not run (not required)

Evidence

- Preconditions: HEAD, merge base from `main` , spec SHA-256 and a clean tree apart from review.md and findings.md all matched the request.
- Delta: 10 commits, 12 files: `.github/workflows/database-backup.yml` , `packages/shared/backups/restore-target.ts` and its test, `packages/shared/scripts/restore-backup.ts` , `packages/shared/package.json` ,

`packages/shared/tsconfig.build.json`, AGENTS.md, the spec, the two plans and the overview (including 8640e41, build-plan item 25 and its overview line; checked against feature 6's decision 4 in `blueprint/history/features/06-confirmations.md`).

- Workflow read line by line: no third-party actions, `contents: read`, secrets scoped to one step and never echoed, apt key fingerprint pinned, `PGSSLMODE=require`, passphrase on a file descriptor, size and table-count refusal, folder choice from the bucket's newest names, minute-stamped keys under the lock.
- Restore read line by line: name regex before any SQL, host and `_dev` guard on both URLs, query or hash refused, tools checked before anything is opened or dropped, drop only after a successful decrypt, pg_restore given separate flags with `PG*` stripped, decrypted file removed in `finally` and leftovers removed at start; no secret is printed.
- Overview fingerprint recomputed by `/overview`'s contract at 24ba21a, 8640e41 and HEAD: the first two match their markers, HEAD's does not (F-451).
- No `.only`, `.skip` or placeholder tests in the delta.

Findings

- New: F-450 [P3] open, F-451 [P3] open, F-452 [P3] open, F-453 [P3] open, F-454 [P3] open
- Closed this pass: F-438, F-439, F-440, F-441, F-442, F-443, F-445, F-446, F-447, F-448
- Left `fixed` with a note: F-444 (the run evidence was not re-observed), F-449 (its repair introduced F-453)
- No P0 or P1 finding is open or fixed

Remaining risk

- `/check` was not run (not required by the request).
- GitHub Actions runs, R2 contents, the bucket lock and lifecycle rules, and the live database were not observed by this review (out of bounds); the runs, the refused delete, the refused insert and the restore drill are taken from the recorded evidence.
- The first scheduled run happens only after the merge, and the first lifecycle expiry under a lock rule of the same age (`6h/`, 3 days) has not happened yet.
- F-437 stays accepted: the public TCP proxy also admits the live superuser by password.
- The restore script's decrypt, drop, restore and count path has no automated test; it rests on the recorded drill.